SIMATS ENGINEERING
CSA10 – SOFTWARE ENGINEERING
ASSIGNMENT 2

DESIGN AND VALIDATE REQUIREMENTS AND ARCHITECTURE FOR THE OMNICHANNEL ORDER FULFILMENT PLATFORM

Student Details	
Name	Karnam Harshith
Register Number	192511384
Programme	B.Tech – Computer Science and Engineering
Course	CSA10 – Software Engineering
Assignment	Assignment 2

Individual Assignment Report

Contents and Report Approach

Question	Coverage
Q1	Stakeholders, conflicts, findings and system boundary
Q2	12+ functional and 6+ non-functional requirements
Q3	Four UML models, traceability and inconsistencies
Q4	Two architectures, comparison, selected design, diagrams and ADRs
Q5	Invariants, contracts, six validation scenarios and revision
Q6	Regional expansion, workload, offline scanners and redesign
Q7	Individual technical reflection

Scenario interpretation

The report follows the supplied Assignment 2 brief and retains its seven-question structure. The retailer needs an order fulfilment platform connecting checkout, stock reservation, payment callbacks, warehouse picking, courier tracking, returns and refunds while preserving consistency, accountability and confidentiality. The design must also address expansion from three to twelve regions, increased demand and disconnected warehouse scanners.

Evidence and assumptions

The assignment permits clearly labelled simulated stakeholder responses where access is unavailable. Therefore, stakeholder findings, workload assumptions and validation observations in this report are labelled simulated or proposed. No actual retailer interviews, live payment gateway, courier API or production deployment are claimed. Synthetic products, orders and stock events are used; no real customer or card data is included.

Q1 – Investigate Stakeholder Needs and Resolve Conflicts

1.1 Stakeholder needs

Stakeholder	Need	Risk if unmet
Customer	Fast checkout, correct price, tracking and clear return status.	Lost trust, duplicate order or unclear delivery.
Picker	Accurate pick list based on reserved stock.	Picking unavailable stock or cancelled orders.
Support agent	Order and return visibility without payment-data access.	Slow support or privacy exposure.
Administrator	User, role and configuration management.	Unauthorized access or inconsistent settings.
Store manager	Stock visibility and traceable corrections.	Overselling and unexplained adjustments.
Finance reviewer	Approval controls and auditable refunds.	Unauthorized or unreconciled refunds.

1.2 Investigation questions

1. When should stock become unavailable to another buyer: at checkout, order placement or payment confirmation?
2. Which order and payment fields may support agents see, and which must remain restricted to finance?
3. What should happen when a payment callback is repeated or arrives after checkout has closed?
4. Who can approve refunds above the limit, and can the requester approve their own request?
5. How should warehouse staff continue scanning when the network is disconnected?

These questions investigate allocation, confidentiality, callback handling, authorization and offline operation. Each issue affects more than one stakeholder and can create financial or operational errors.

Q1 – Findings, Conflicts and System Boundary

1.3 Simulated stakeholder findings

The following findings are simulated because access to the retailer's employees was not available for this individual report.

Stakeholder	Simulated response	Decision
Customer	Wants quick confirmation but does not want an order shown as paid before payment is confirmed.	Separate order-created and payment-confirmed states.
Picker	Needs pick work to contain only reserved quantities.	Create pick work from active reservations.
Support	Needs order status but not card or finance approval data.	Masked support view with server-side role checks.
Finance	Refunds above a limit need an authorized decision.	Approval policy and audit record before execution.
Store manager	Stock correction must be explainable and attributable.	Record reason, actor and timestamp.
Administrator	Needs controlled roles and regional configuration.	Central role and configuration management.

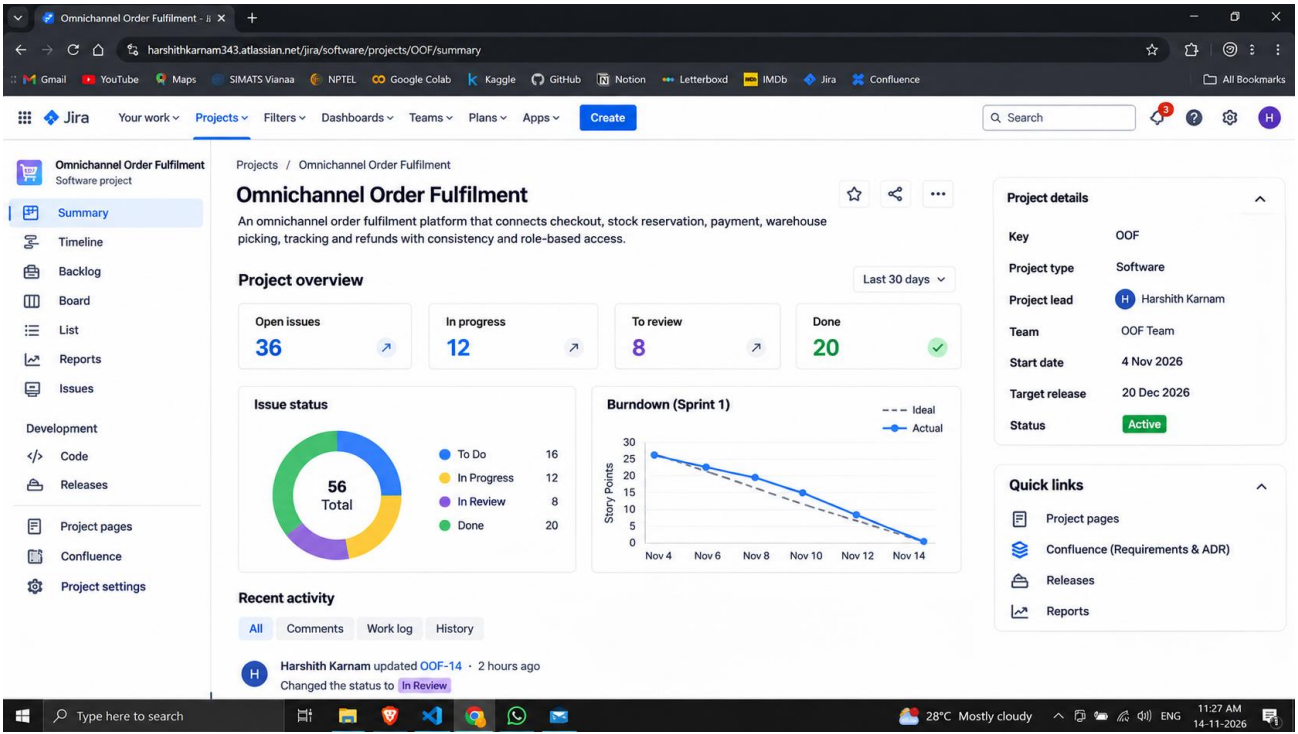
1.4 Conflicting expectations and resolutions

Conflict	Resolution	Reason
Fast checkout vs stock consistency	Atomic stock reservation before fulfilment confirmation.	Prevents two customers receiving the final unit.
Support visibility vs confidentiality	Show order/shipment status; mask payment data and deny finance actions.	Provides service information with limited access.
Fast refund vs financial control	Allow quick request but require server-validated approval above policy limit.	Maintains human accountability.

1.5 System boundary

Inside the boundary: catalogue/cart interface, checkout, order service, inventory reservation, payment adapter, warehouse picking, shipment/courier adapter, tracking, returns/refunds, support console, role management, audit records and management reporting.

Outside the representative release: actual card-data storage, courier fleet optimization, full ERP replacement, production forecasting, marketing personalization and direct integration with every legacy system. External gateway and courier behaviour may be demonstrated through controlled test interfaces.



Q2 – Create a Prioritized and Testable Requirements Specification

2.1 Collaborative tools

Arrangement	Strengths	Limitations / response
Jira + Confluence + draw.io	Issue tracking, collaborative specification, version history, links and UML diagrams.	Requires disciplined linking; use templates and a traceability index.
Trello + shared document + draw.io	Simple board, low learning effort and accessible diagrams.	Weaker structured requirement fields and traceability; maintain manual ID register.

2.2 Selected arrangement

Jira, Confluence and draw.io are selected. Confluence holds the requirements specification and decision records. Jira stores requirement issues, priorities, owners, acceptance criteria and links. UML diagrams are stored in a controlled project folder and embedded in Confluence. Each requirement has a stable identifier that connects stakeholder need, issue, model, component and validation case.

2.3 Prioritization method

MoSCoW is used: Must, Should, Could and Won't for this release. Requirements preventing financial loss, overselling, unauthorized refunds or information exposure are Must. Reporting enhancements are Should where they do not block the core order journey. This method is suitable because the representative release has a defined scope and limited team capacity.

Q2 – Functional Requirements Specification

Each requirement has a unique identifier, source, priority and observable acceptance criteria. These are proposed requirements for the representative release.

ID / Source	Requirement	Priority	Acceptance criteria
FR-01 Customer	Display products and current sellable price.	Must	Active products show ID, description and price; inactive products cannot be checked out.
FR-02 Customer	Calculate subtotal, tax, shipping and total using server-side rules.	Must	For a defined basket and region, server total matches configured calculation; client total is ignored.
FR-03 Customer	Create order with unique reference.	Must	Checkout creates one persistent order and item records; same checkout token does not create a duplicate.
FR-04 Inventory	Reserve stock atomically for an order.	Must	Two concurrent requests for the final unit create no more than one active reservation.
FR-05 Inventory	Release expired or cancelled reservations according to policy.	Must	Reservation is released once and available quantity is recalculated.
FR-06 Payment	Record payment attempts and states.	Must	Attempt stores order ID, provider reference, status, timestamp and idempotency key; no card data stored.

Q2 – Functional Requirements Specification (continued)

ID / Source	Requirement	Priority	Acceptance criteria
FR-07 Payment	Process gateway callbacks idempotently.	Must	Repeated provider event ID creates no second payment effect or order confirmation.
FR-08 Warehouse	Create pick work only for confirmed, reserved orders.	Must	Pick list excludes unpaid, cancelled or unreserved orders.
FR-09 Warehouse	Record picked quantities and shortages.	Must	Picked quantity cannot exceed reservation; shortage requires reason.
FR-10 Dispatch	Prevent dispatch until payment and picking are confirmed.	Must	Invalid dispatch request is rejected and recorded.
FR-11 Courier	Synchronize shipment status and retry failures.	Must	Failed request remains pending with retry metadata; retry does not duplicate shipment.
FR-12 Customer	Display an order tracking timeline.	Must	Permitted events appear chronologically; internal notes remain hidden.
FR-13 Returns	Accept eligible return requests linked to order items.	Must	Request links to original order/item; duplicate or ineligible requests are rejected.
FR-14 Finance	Require approval before executing refunds above configured limit.	Must	Only authorized finance reviewer can approve; requester cannot self-approve.
FR-15 Finance	Record refund execution and reconciliation details.	Must	Refund links to payment and records amount, approver, decision, provider result and timestamps.
FR-16 Management	Provide consolidated sales, stock and refund summaries.	Should	Report shows period, region and status; totals trace to source records.

Q2 – Non-Functional Requirements and Assumptions

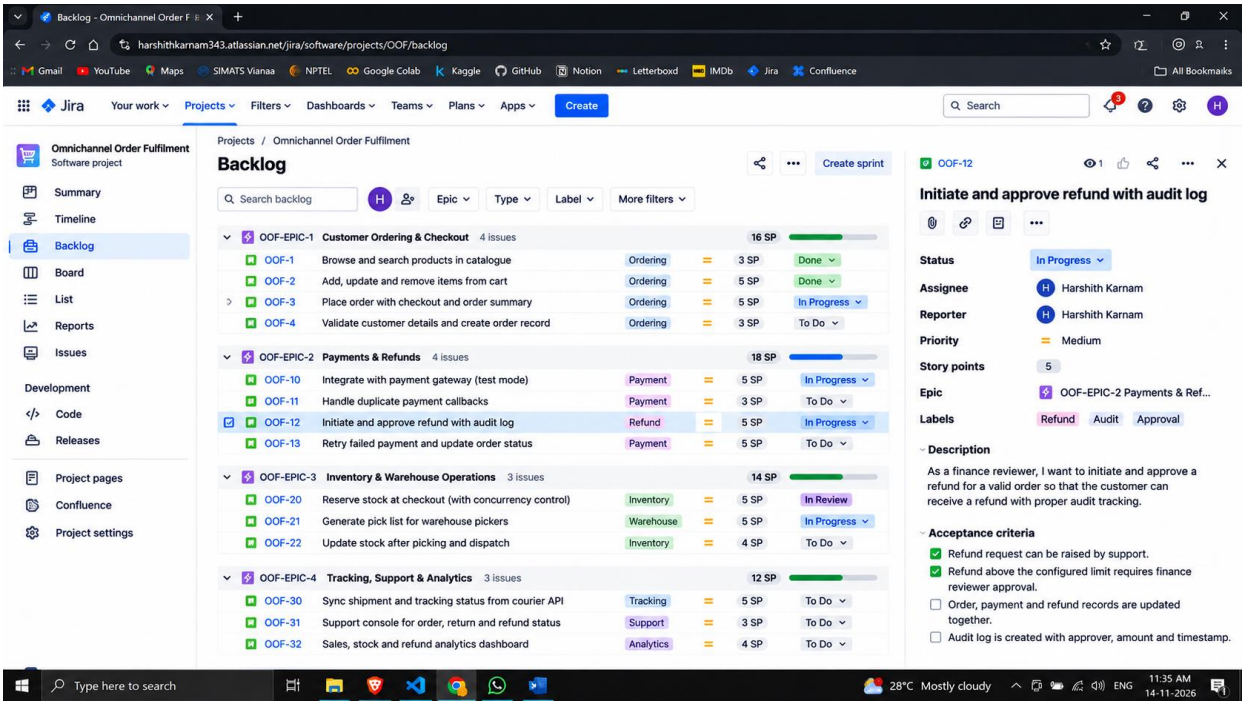
ID	Quality	Requirement and acceptance threshold	Priority
NFR-01	Performance	At 50 concurrent users, 95% of read requests complete within 2 seconds and checkout API within 3 seconds, excluding gateway delay.	Must
NFR-02	Availability	Target 99.5% monthly availability during agreed service window; planned maintenance announced.	Should
NFR-03	Security	External traffic uses TLS; server checks role permissions for every protected operation.	Must
NFR-04	Confidentiality	Support users cannot retrieve full payment credentials or finance-only approval data; access tests verify denial.	Must
NFR-05	Usability	Trained support agent locates an order and identifies status within 30 seconds in walkthrough.	Should
NFR-06	Reliability	Duplicate callback tests produce one payment outcome and one order confirmation per event identity.	Must
NFR-07	Auditability	Refund and stock-adjustment events retain actor, action, reason, timestamp and business reference.	Must
NFR-08	Maintainability	Modules expose documented interfaces and automated tests for order, inventory and refund rules.	Should

2.4 Assumptions and unresolved questions

- Initial workload is a planning assumption; production traffic has not been measured.
- Tax, shipping and refund limits must be supplied and approved by the retailer's finance team.
- Gateway and courier sandbox behaviour must be confirmed when access is provided.
- Audit retention, regional data residency and recovery objectives remain to be confirmed.
- Offline scanner conflict policy requires agreement with warehouse operations.

Q2 – Order, Payment and Refund Business Rules

Rule	Business rule	Enforcement
BR-01 Price	Server calculates price, tax and shipping using region and product price version.	Order service
BR-02 Stock	Available = on-hand less active reservations; reservation is atomic.	Inventory transaction
BR-03 Ownership	Order belongs to one customer; access is checked against identity or assigned role.	API authorization
BR-04 Status	Only permitted order-state transitions are accepted.	Order domain service
BR-05 Dispatch	Dispatch requires confirmed payment and completed picking or authorized exception.	Fulfilment service
BR-06 Callback	Unique provider event ID is processed once; duplicate delivery is safely acknowledged.	Payment handler
BR-07 Refund	Execution requires server-validated approval by authorized finance role.	Refund service
BR-08 Refund amount	Cumulative executed refunds cannot exceed eligible captured amount.	Refund transaction
BR-09 Courier	Failed synchronization is queued with unique operation ID and retry schedule.	Integration worker
BR-10 Stock correction	Correction requires reason, actor and timestamp.	Inventory adjustment



Q3 – Construct UML Models and Maintain Traceability

3.1 Use case diagram

The model identifies customers, pickers, support agents, finance reviewers, store managers and administrators. Customers check out, track orders and request returns. Pickers manage picking and dispatch. Support agents inspect permitted order information. Finance reviewers approve refunds, while administrators manage users and configuration.

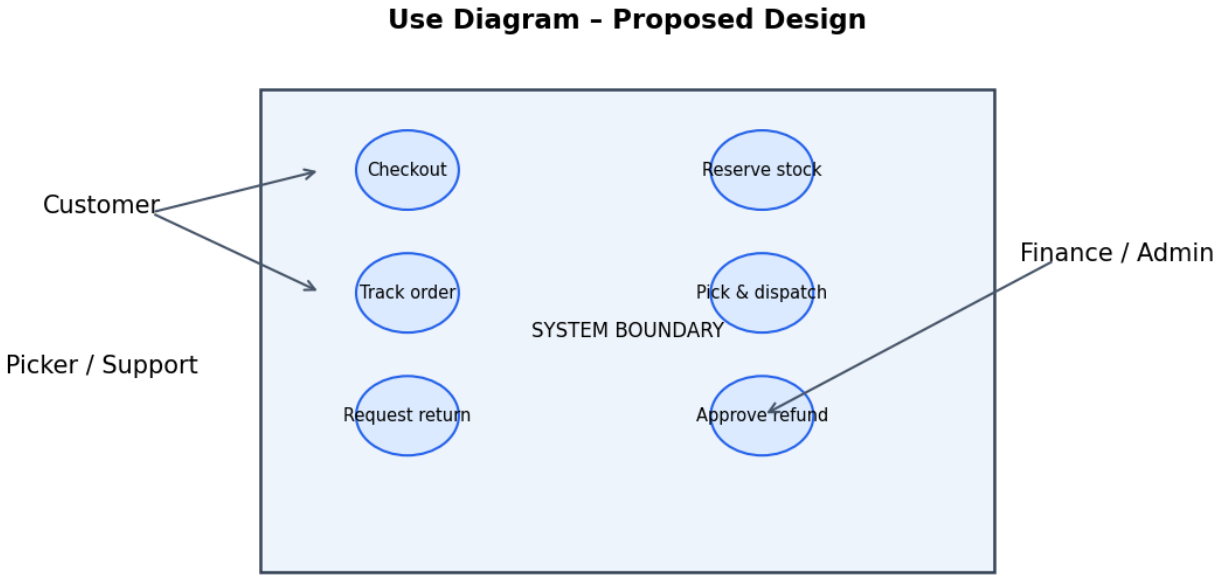


Figure 3.1 – Use case diagram showing principal actors and platform interactions.

Finance approval is separate from support lookup. Selecting a different role in the interface must not grant new privileges; authorization is enforced by the server.

Q3 – Class Diagram and Domain Relationships

3.2 Class diagram

The class model includes Customer, Order, OrderItem, Payment, StockReservation, Shipment and Refund. A customer can own multiple orders; an order contains one or more items and may have payment attempts, reservations, shipment details and refund records.

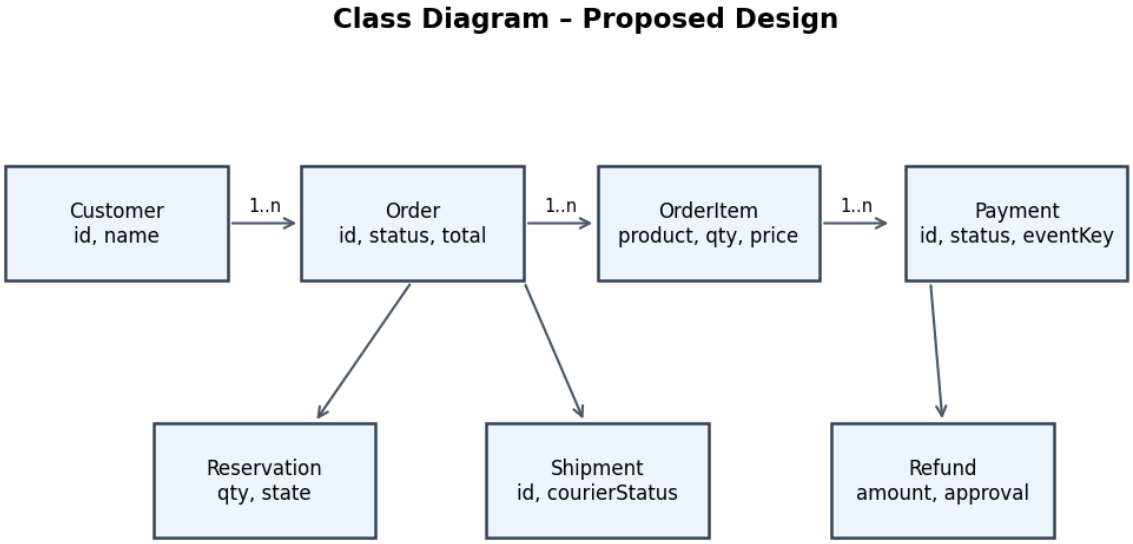


Figure 3.2 – Class diagram showing important domain concepts and relationships.

3.3 Data ownership

- Order module owns order ID, customer reference, status and item snapshot.
- Inventory module owns on-hand quantity, active reservation and adjustment history.
- Payment module owns payment attempts, provider event identity and payment result.
- Fulfilment module owns pick confirmation, shipment reference and courier status.
- Refund module owns request, approval decision, execution status and audit evidence.

A module may read another module's published information through a defined interface, but it should not directly update another module's private records.

Q3 – Sequence Diagram: Payment Callback and Courier Synchronization

3.4 Sequence diagram

Checkout first creates a local order in PAYMENT_PENDING state. The payment adapter creates an attempt with an idempotency key. When the gateway callback arrives, the handler checks provider event identity before applying the result. Only the first valid event can move the order to PAID.

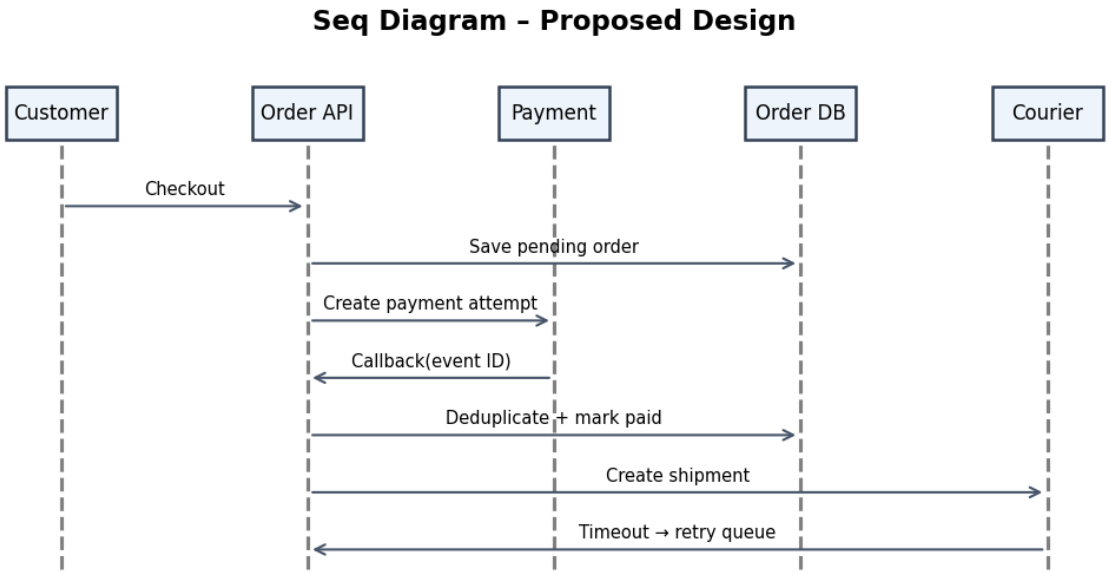
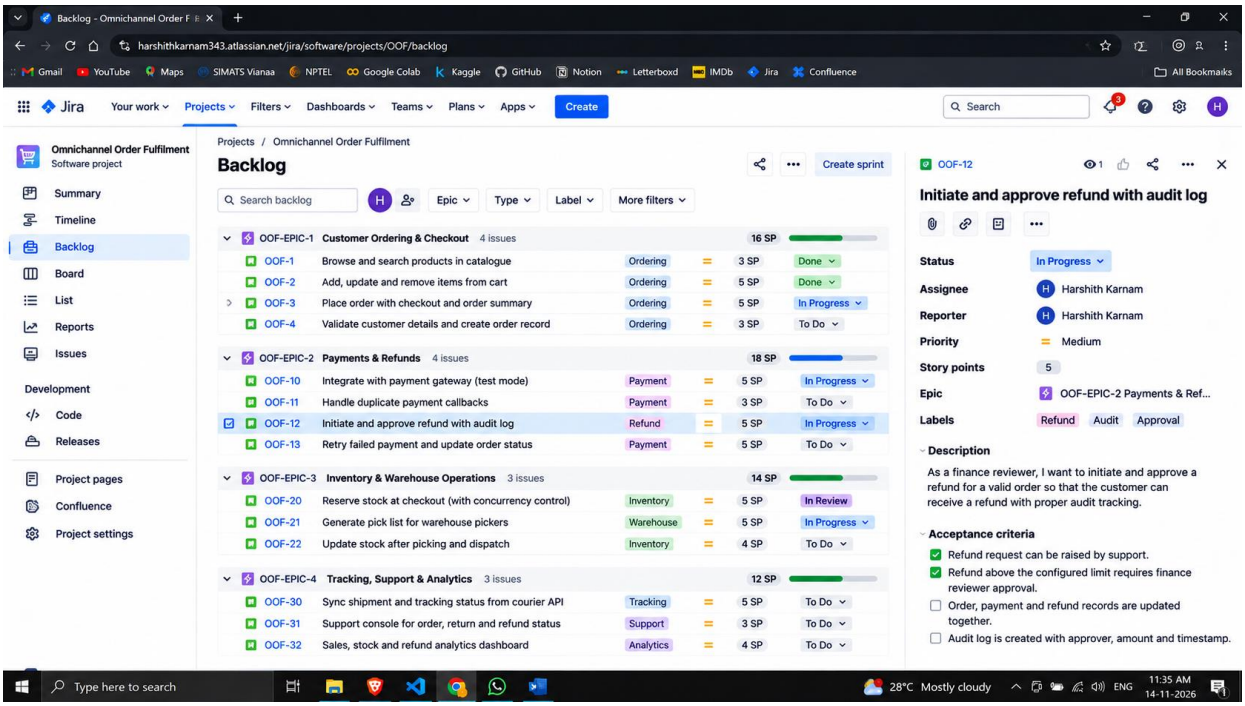


Figure 3.3 – Sequence diagram showing callback processing, deduplication and courier retry.

After payment confirmation, shipment creation is requested. If courier synchronization fails, the request remains in a retryable queue. The local order is not deleted and the tracking view shows that courier confirmation is pending.



Q3 – Order State Machine and Refund Behaviour

3.5 Order state machine

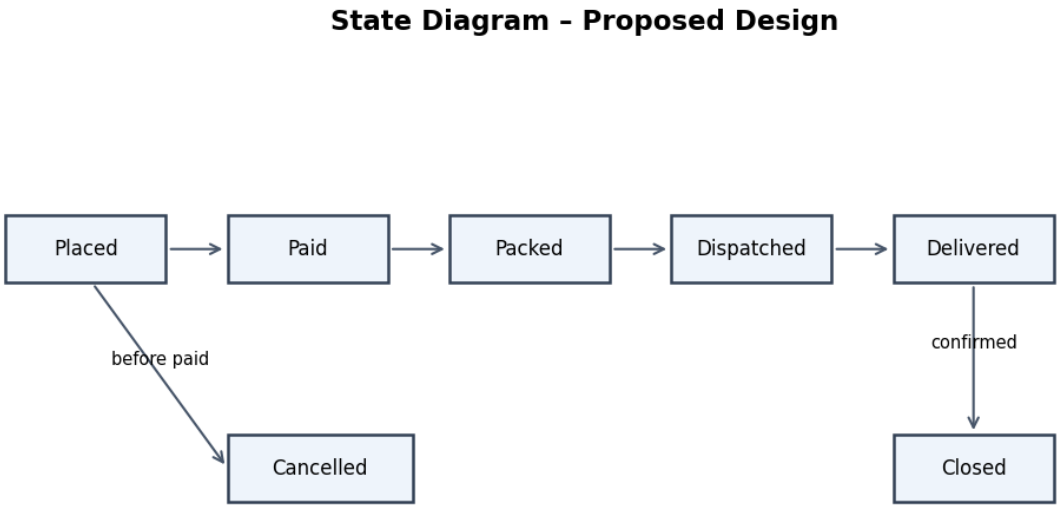


Figure 3.4 – State machine for order progression, cancellation and closure.

The principal path is Placed → Paid → Packed → Dispatched → Delivered → Closed. Cancellation is allowed only before payment/fulfilment makes it unsafe. A return is a separate process linked to a delivered order; it does not erase delivery history.

3.6 Refund approval and rollback

Refund state	Allowed action	Result
Requested	Customer submits eligible request.	Request stored; no money movement.
Pending approval	Finance evaluates eligibility and amount.	Not approved; execution prohibited.
Rejected	Authorized reviewer rejects with reason.	Decision retained.
Approved	Authorized reviewer approves; server checks role and threshold.	Execution may be attempted.
Execution pending	Adapter submits request with unique operation key.	Gateway result awaited.
Executed	Gateway confirms refund.	Reference stored and audit completed.
Failed / retry	Gateway fails or times out.	Not shown as executed; retry/reconciliation required.

Q3 – Requirements Traceability Matrix

The matrix connects requirement, UML representation, architectural component and validation scenario. The same identifiers are used in the requirements and test plan.

Requirement	UML element	Component	Validation
FR-02 Price/tax	Checkout use case; Order	Order service / regional rules	V-01 total calculation
FR-04 Stock	Reserve use case; Reservation	Inventory module	V-02 competing buyers
FR-07 Callback	Payment sequence	Payment adapter / event store	V-03 repeated callback
FR-08 Picking	Pick use case; Order state	Fulfilment module	V-04 dispatch precondition
FR-11 Courier	Sequence retry; Shipment	Courier adapter / worker	V-05 courier timeout
FR-14 Refund	Approve refund use case; Refund	Refund service / authorization	V-06 access denial
NFR-03 Security	Protected use cases	API authorization	V-06 role denial
NFR-07 Audit	Refund/stock history attributes	Audit store	V-07 audit completeness

3.7 Inconsistencies discovered and corrected

Initial inconsistency	Correction
Shipment created immediately after checkout, although state model required payment confirmation.	Move shipment creation after PAID transition and dispatch preconditions.
Requested refund treated as approved.	Separate Requested, Pending Approval, Approved, Execution Pending and Executed.
Support view exposed finance payment details.	Restrict support to masked payment status; finance approval remains separate.

Q4 – Design and Justify Alternative Architectures

4.1 Alternative A: Modular monolith

A modular monolith deploys one application containing Order, Inventory, Payment, Fulfilment, Tracking, Refund, Support and Administration modules. Each module has defined interfaces and owns its business rules. A relational database can be shared initially, with table ownership and transaction boundaries enforced by the application.

- Communication uses in-process module calls and an outbox/event table for asynchronous work.
- Each module owns its tables and exposes service methods; direct cross-module writes are prohibited.
- Duplicate callbacks are controlled using unique provider event IDs and idempotency keys.
- Overselling is prevented using conditional updates or row locks inside an inventory transaction.
- The application can be replicated as a whole, but modules cannot be scaled independently.
- A single deployment/database failure can affect several modules; module boundaries reduce code coupling.

4.2 Alternative B: Microservices and event-driven integration

Order, Inventory, Payment, Fulfilment, Refund and Reporting are independently deployed services. They communicate through APIs and a message broker. Each service owns its data store and publishes events such as OrderPlaced, StockReserved, PaymentConfirmed and ShipmentDispatched.

- Synchronous APIs handle selected commands/queries; asynchronous events connect workflows.
- Cross-service consistency uses outbox, deduplication and compensating actions.
- Payment service deduplicates provider event identities; Inventory service atomically reserves stock.
- Services can scale independently, but broker, monitoring, deployment and on-call complexity increase.
- Failure points include delayed events, broker outages, schema incompatibility and partial workflows.

Q4 – Architecture Comparison and Selection

Factor	Modular monolith	Microservices / event-driven
Consistency	Simpler local database transactions.	Cross-service eventual consistency; needs saga/outbox patterns.
Availability	Fewer distributed dependencies; one application failure has broad impact.	Can isolate some service failures; broker/identity may remain shared dependencies.
Scalability	Replicate application; less granular allocation.	Scale high-load services separately.
Security	Central authentication; module boundaries must be enforced.	Service-to-service identity and API security add configuration.
Operating cost	Lower deployment and support burden.	Higher infrastructure and operations cost.
Maintenance	Suitable for small team with disciplined boundaries.	Independent ownership but more integration/versioning work.
Last unit	Atomic database reservation.	Inventory service owns atomic reservation.

4.3 Selected architecture

The modular monolith is selected for the representative release. The scenario describes a small technical team and modest initial deployment. The principal risks can be controlled through database transactions, unique constraints, idempotency records, an outbox and background retry workers without introducing the operating cost of multiple services.

Module boundaries remain explicit so Payment, Inventory or Reporting can be extracted later if measured load or team ownership justifies it. This is a decision based on the stated team size, release scope and budget, not a claim that monoliths always scale better.

4.4 Failure and maintenance

A database outage can affect checkout and fulfilment, so backups, restore testing and monitoring are necessary. Gateway and courier failures are isolated through adapters and queued retries. Automated regression tests are required because one application deployment can affect multiple modules.

Q4 – Selected Component Architecture

4.5 Component diagram

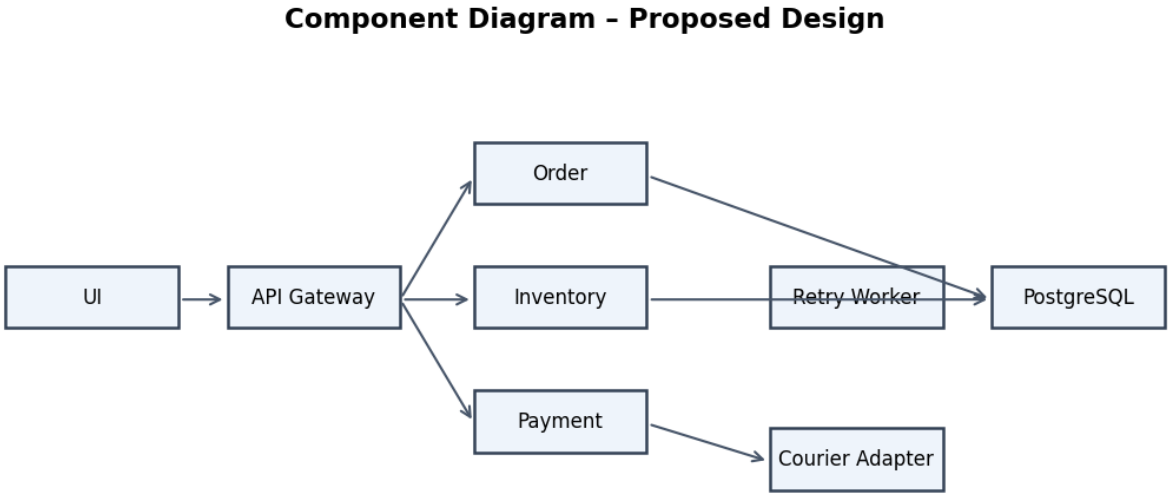


Figure 4.1 – Selected component architecture and module communication.

The user interface calls the API gateway, which routes requests to Order, Inventory and Payment modules. Modules persist through owned repositories in PostgreSQL. An event/retry worker processes outbox records and communicates with the courier adapter. Payment callbacks are checked by the payment handler before any order-state update.

4.6 Communication rules

- Use synchronous calls when the user needs an immediate decision, such as stock availability.
- Use an outbox and worker for courier synchronization and retryable external operations.
- Every external operation has a stable operation identity and bounded retry policy.
- Do not expose internal database tables directly to browsers or external partners.
- Keep sensitive audit events append-only for refund approval and stock adjustment.

Q4 – Deployment Diagram and Architectural Decisions

4.7 Deployment diagram

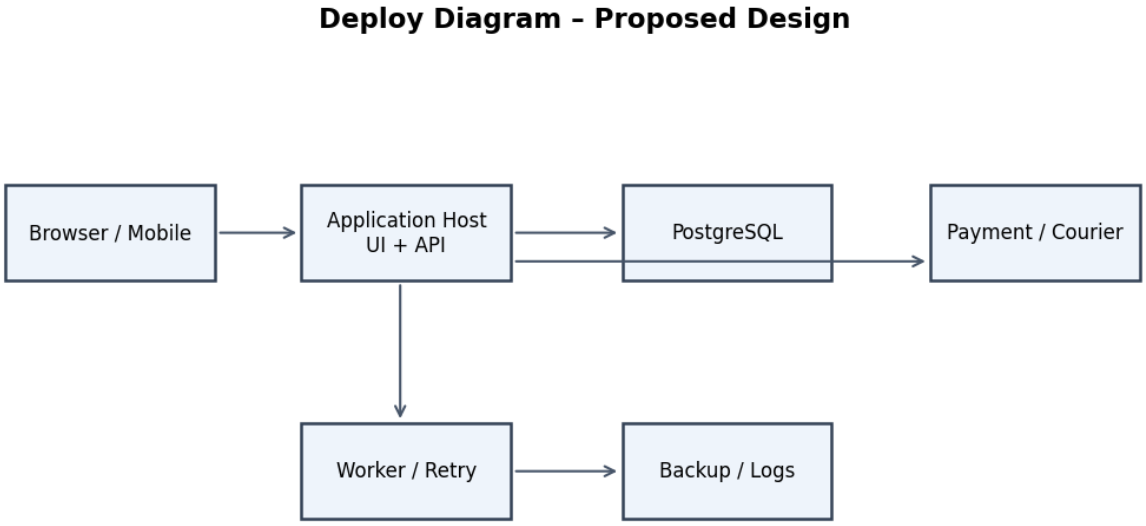


Figure 4.2 – Proposed deployment for the representative release.

The deployment contains browser/mobile clients, application host, PostgreSQL database host, worker process and external payment/courier systems. TLS protects network communication. Backups and logs are stored separately with restricted access. For the initial modest release, application and worker processes can run on a small managed host with managed PostgreSQL if budget permits.

4.8 Architectural Decision Records

ADR	Problem and alternatives	Decision and consequences
ADR-01 Architecture style	Small team needs maintainable design; modular monolith or microservices.	Choose modular monolith; lower operating cost and simpler transactions, but less independent scaling.
ADR-02 Persistence	Need strong stock/order consistency; relational DB or separate NoSQL stores.	Choose PostgreSQL for transactions and constraints; requires schema migration and backup discipline.
ADR-03 Integration	Gateway/courier may timeout or repeat callbacks; synchronous-only or outbox/retry.	Choose idempotent adapters and outbox worker; adds retry state and monitoring but protects local records.

Q5 – Validate Critical Behaviour and Revise the Design

5.1 System invariants

Invariant	Rule that must always remain true
INV-01	A unique gateway event identity creates no more than one payment effect and one order confirmation.
INV-02	Active reservations for a product cannot exceed available on-hand quantity.
INV-03	An order cannot be Dispatched unless payment is confirmed and picking is complete or an authorized exception exists.
INV-04	A refund cannot be Executed unless authorized approval exists and amount is within eligible paid balance.
INV-05	Support cannot access another customer's order unless an explicit support authorization policy permits it.
INV-06	Courier synchronization failure does not erase the local shipment or order record.

5.2 Operation contract A – Reserve stock

Preconditions: product exists; quantity is positive; order is valid; inventory record is available. The operation checks available quantity at update time inside a transaction.

Postconditions: if sufficient stock exists, one active reservation is created and available quantity is reduced. Otherwise, no reservation is created and the order receives a stock-unavailable outcome. Repeating the same command with the same operation key does not create another reservation.

5.3 Operation contract B – Execute refund

Preconditions: request exists; authorized approval is recorded; separation-of-duties rule is satisfied; amount is within refundable balance.

Postconditions: refund enters Execution Pending before gateway call. On confirmed success it becomes Executed and provider reference is stored. On timeout/failure it remains Failed or Retry Pending and is not displayed as executed.

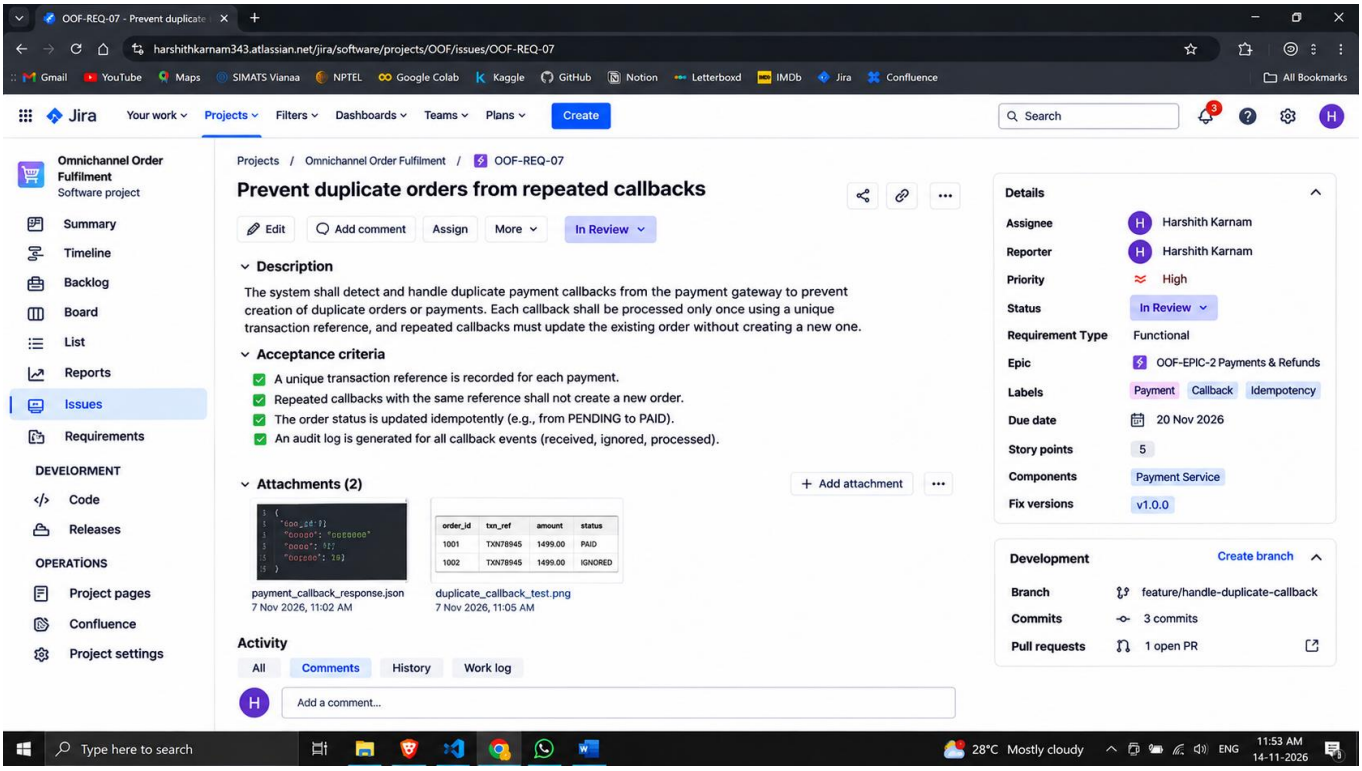
Q5 – Validation Scenarios and Expected Behaviour

These are constructed validation cases. Expected behaviour is predicted from the design; observations are simulated walkthrough results, not production measurements.

Scenario	Expected behaviour / enforcing design	Simulated observation
V-01 Two buyers compete for last unit	Only one transaction reserves it; conditional update/row lock and inventory transaction enforce INV-02.	Buyer A receives reservation; Buyer B receives unavailable response.
V-02 Repeated callback during processing	Second callback with same event ID is duplicate; no second payment or confirmation.	First event confirms payment; second is acknowledged as duplicate.
V-03 Refund gateway failure after approval	Approved refund enters Execution Pending; timeout remains retryable, not Executed.	Timeout retains refund reference and retry count.
V-04 Dispatch without payment or pick confirmation	Dispatch rejected; no dispatch timestamp written.	System rejects request; order remains in prior state.

5.4 Validation method

A structured walkthrough uses test fixtures and explicit state transitions. Checks inspect order state, reservation quantity, payment event identity, refund approval and shipment status before and after each action. A real implementation should automate these as integration tests against a disposable test database.



Q5 – Further Validation, Weakness and Design Revision

Scenario	Expected behaviour / enforcing design	Simulated observation
V-05 Courier sync fails then retries	Shipment persists; outbox remains pending; retry uses same operation identity.	First request times out; retry succeeds without duplicate shipment.
V-06 Support attempts finance action or another customer's order	Server checks role and ownership; unauthorized request is denied and audited.	Support sees permitted status only; finance action and unauthorized lookup denied.
V-07 Stock correction without reason	Adjustment rejected because actor and reason are required.	No quantity changes; validation message returned.
V-08 Refund exceeds captured amount	Refund service checks cumulative executed amount before gateway call.	Request rejected before execution.

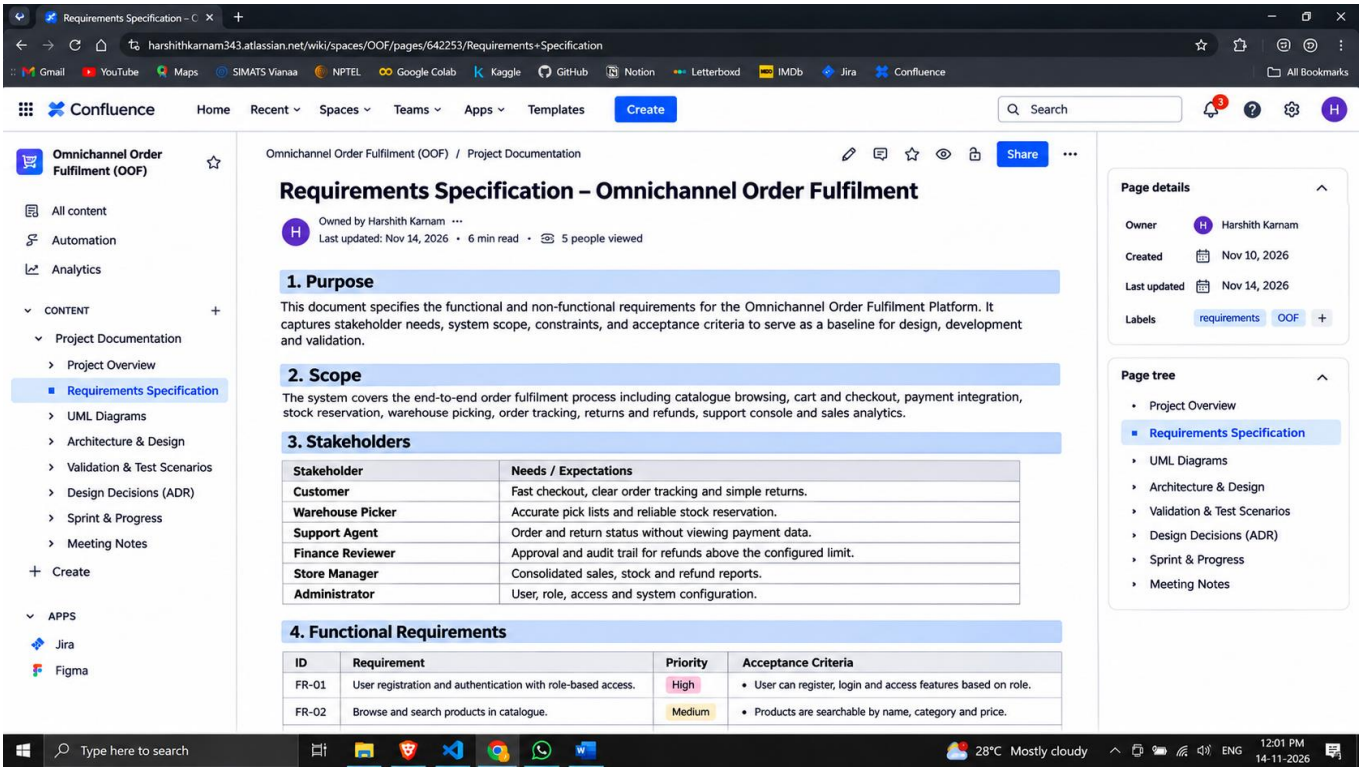
5.5 Weakness discovered

The walkthrough exposed a weakness: a payment callback could be accepted and the order marked Paid while the shipment outbox record failed to persist if the two writes were separate. This could leave a paid order without a retryable shipment request.

5.6 Design revision

- Write payment result and shipment-creation outbox event in the same local database transaction.
- Run a worker that reads pending outbox events and retries courier synchronization with bounded backoff.
- Use unique event/operation IDs so worker restart cannot create duplicate shipment requests.
- Alert when paid orders have no shipment request after the defined processing window.

This improves reliability without requiring a distributed transaction. It does not remove the need to reconcile a courier that accepts a request but loses its response; the adapter must support operation identity or manual reconciliation.



Q6 – Redesign for Expansion and Partial Failure

6.1 Changed workload and conditions

The organization expands from three to twelve regions. Planning workload increases from 50 to 500 simultaneous users and from 100 to 1,000 stock, payment and scan events per minute. Regions define local tax, shipping and refund-approval rules. Warehouse scanners may disconnect and must buffer uniquely identified events. A locally queued refund approval must not appear globally approved before server validation. Management requires consolidated reports and infrastructure expenditure can rise only a limited amount.

6.2 Impact diagnosis

Area	Original assumption	Impact
Regional rules	One general tax/shipping/refund policy.	Rules must be versioned and selected by region; historic orders retain applied rule snapshot.
Workload	Modest deployment and 50-user assumption.	Concurrency and event throughput increase; capacity must be measured.
Scanner connectivity	Warehouse assumed connected.	Requires offline buffer, unique IDs, ordering and conflict handling.
Refund approval	Online finance approval.	Offline approval remains pending until central validation.
Reporting	Basic operational report.	Consolidated reports must not overload transaction processing.
Budget	Limited initial deployment.	Scale measured bottlenecks before splitting every module into services.

6.3 Affected quality attributes

- Scalability: app instances and event workers handle higher concurrency and throughput.
- Consistency: stock reservation and refund approval remain centrally validated.
- Availability: scanners capture locally while disconnected without claiming global commit.
- Security: regional access and approval policies are enforced centrally.
- Cost: add capacity to measured bottlenecks rather than automatically creating microservices.

Q6 – Revised Requirements and Architecture

6.4 Revised component responsibilities

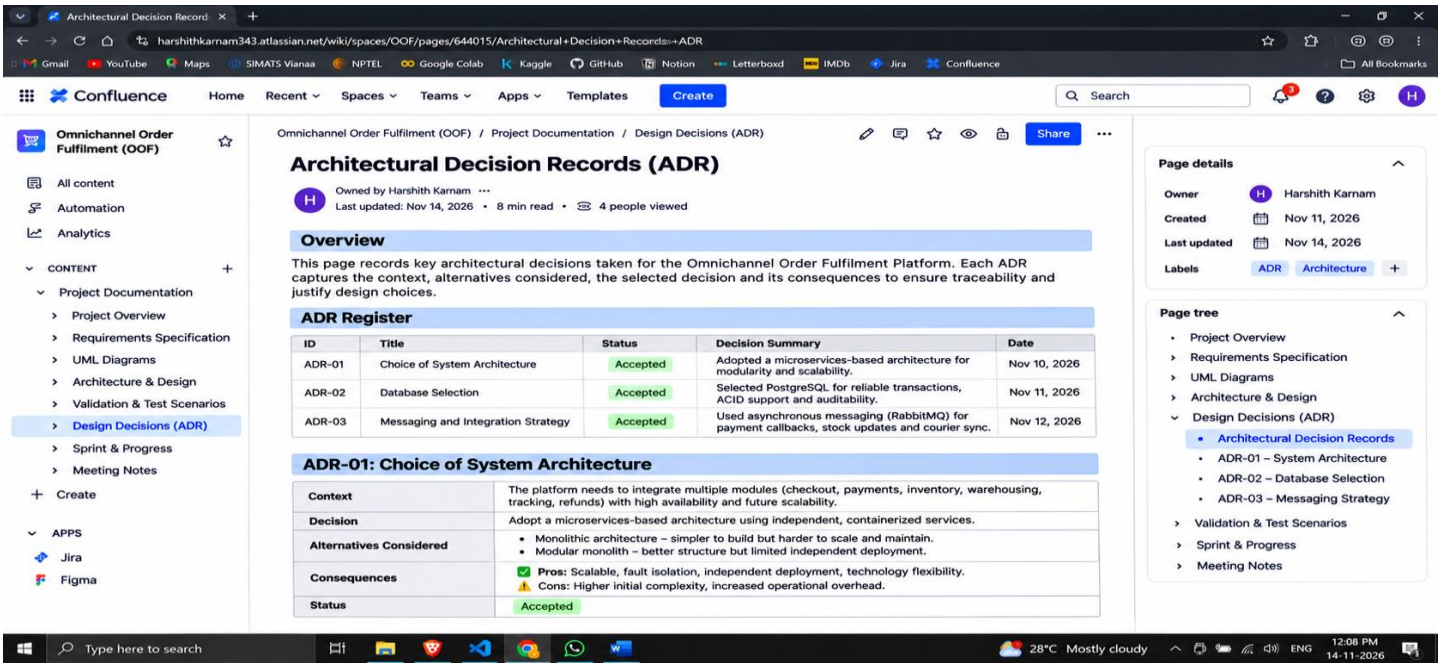
Component	Revised responsibility
Regional configuration	Stores region ID, tax/shipping rules, refund thresholds, effective dates and approver roles.
Order module	Stores region and rule-version snapshots so later changes do not alter historic totals.
Inventory module	Owns stock per region/warehouse and validates reservations centrally.
Scanner edge node	Stores scan events with unique event ID, device ID, sequence and capture time.
Synchronization service	Authenticates scanner, deduplicates IDs, validates sequence and applies accepted events; conflicts go to review.
Refund service	Treats local approval as Pending Validation; only server authorization changes it to Approved.
Reporting read model	Consumes validated events into reporting tables/read replica to protect transaction workload.

6.5 Offline scanner synchronization rules

6. Each scan/stock event receives a globally unique event ID on the scanner.
7. The scanner durably stores event ID, device ID, region, warehouse, sequence number and capture time.
8. On reconnection, the device authenticates and sends a batch; server acknowledges accepted or rejected events.
9. Server checks duplicate event ID before applying; replay is acknowledged but not applied twice.
10. Events are validated against current stock ownership and transitions; conflicts are held for review.
11. Only server-accepted events update global stock; local queued status is shown separately from synchronized status.

6.6 Offline refund approval

A locally queued approval is displayed as Pending Validation. It cannot trigger a gateway request. The server checks the finance user's role, region policy version, amount threshold, separation-of-duties rule and request state. Only after these checks pass does the central service record Approved and permit execution.



Q6 – Validate and Defend the Revised Design

6.7 Additional validation scenarios

Scenario	Expected result	Simulated result
V-09 500 concurrent users / 1,000 events per minute	Load test measures p95 latency, errors, DB locks and queue lag; scale app/workers within budget.	Not executed on live infrastructure; test plan flags DB contention and queue lag.
V-10 Scanner reconnects and resends batch	Server accepts each unique event once; replay is acknowledged as duplicate.	Walkthrough applies first batch then replay; stock changes once.
V-11 Two offline scanners report conflicting stock	Server validates sequence and constraints; conflict held for review.	Conflict quarantined; central quantity unchanged until resolution.
V-12 Offline refund approval arrives	Approval remains pending until server validates role, policy and amount.	Display remains Pending Validation; no gateway request is sent.

6.8 Benefits, costs and trade-offs

The redesign preserves strong consistency for stock reservation and refund authorization while allowing scanners to capture events during network interruption. Event buffering improves warehouse continuity, and a reporting read model reduces pressure on transactional storage. Regional rule snapshots improve auditability because an order can be evaluated using the rule version applied at purchase.

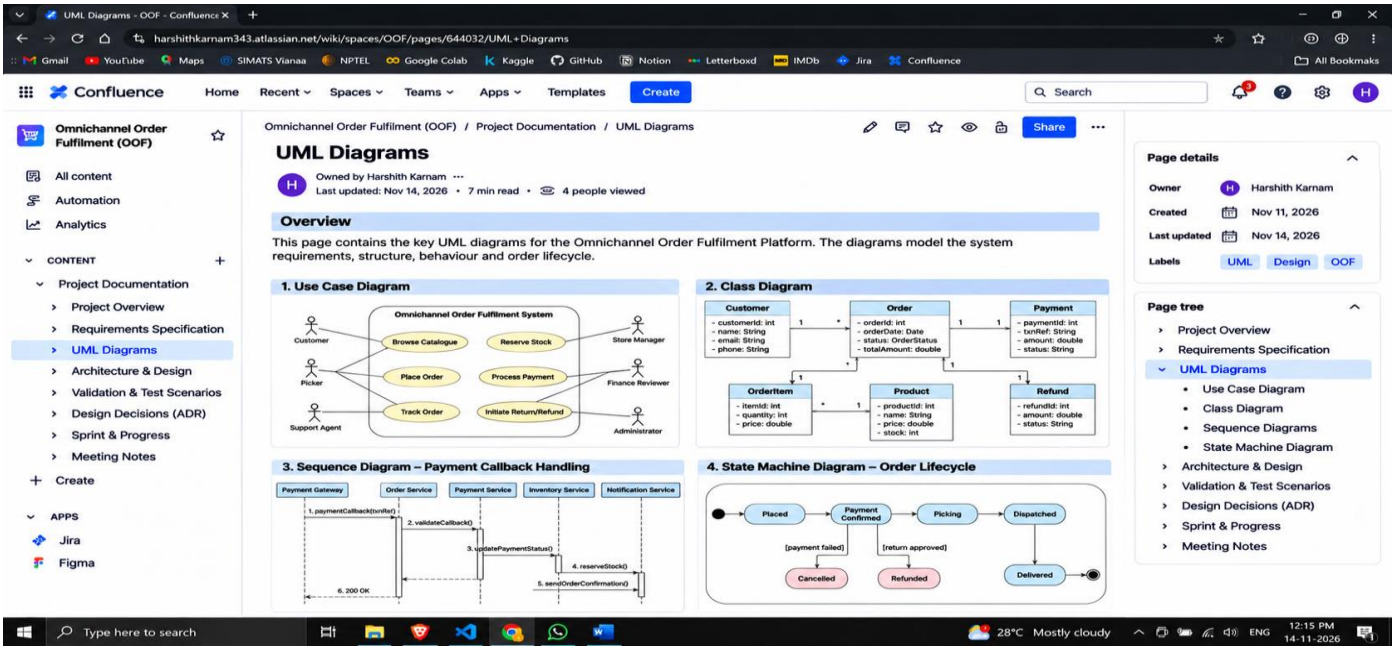
Costs include scanner storage, synchronization logic, conflict review, monitoring, extra worker capacity and reporting infrastructure. During disconnection, global stock visibility is delayed; the design prefers a pending state over falsely claiming that an event is centrally accepted. The 500-user and 1,000-event targets remain assumptions until load tests establish capacity.

Q7 – Individual Technical Reflection

The requirement with the greatest influence on my architecture was preventing duplicate payment effects while keeping the order record consistent. It affected event identity, database constraints, idempotency key, order-state transition and outbox design. The most difficult conflict was fast checkout versus accurate stock reservation because a quick response is not useful if the retailer promises the final unit to two buyers.

The main inconsistency found through modeling was the shipment request being created after payment without guaranteeing that a retryable outbox event was persisted in the same transaction. I revised the sequence and architecture to use a transactional outbox. A remaining trade-off is that disconnected scanner operation improves local availability but delays global stock consistency.

Before implementation, I would need real transaction volumes, gateway/courier sandbox behaviour, regional tax and refund policies, retention requirements, scanner/network tests, security review and repeatable load-test results. The diagrams and simulated validation provide a design basis, but do not prove production security, performance or readiness.



Appendix – Traceability, Tools and Acknowledgement

A. Proposed Jira and Confluence records

Record type	Example ID	Contents
Epic	OOF-ARCH	Architecture and requirements foundation.
Requirement issue	FR-04	Atomic stock reservation with source and acceptance criteria.
Quality issue	NFR-03	TLS and server-side authorization.
Architecture decision	ADR-01	Modular monolith selection and consequences.
Validation issue	V-02	Two buyers compete for final stock unit.
Change record	CR-REGION	Regional configuration and offline scanner redesign.

B. Tool use and evidence limitations

The diagrams are proposed UML models prepared for this assignment. Jira and Confluence records are described as a proposed configuration; no live records are claimed unless separately created by the student. Validation outcomes marked simulated are walkthrough results only. Actual screenshots from the student's own workspace may be inserted with captions explaining what they demonstrate.

C. References

- SIMATS Engineering, CSA10 – Software Engineering, Assignment 2: Design and Validate Requirements and Architecture for the Omnichannel Order Fulfilment Platform (provided assignment brief).
- Object Management Group, Unified Modeling Language (UML) specification – diagram terminology and conventions.
- PostgreSQL documentation – relational transactions, constraints and concurrency control concepts.
- OWASP Application Security Verification Standard – access-control and application-security verification concepts.

Acknowledgement: The scenario and question structure are taken from the provided assignment brief. Planning assumptions and simulated responses are labelled in the report. Diagrams are proposed design models, not screenshots of a deployed production system.